```
// ==========================================
// TIK-TAK-TOE PROJECT (GROUP 25)
// ==========================================

Class TicTacToeApp
    // UI elements & screens
    Frame mainWindow
    CardLayout screenManager
    Panel mainContainer
    Panel menuScreen, tokenScreen, diffScreen, boardScreen

    Button boardButtons[3][3]
    String stateGrid[3][3]
    Label infoLabel, scoreTrackerLabel

    // Game stats tracker
    Int winsX = 0, winsO = 0, ties = 0
    Boolean playerXTurn = true
    Boolean isGameRunning = true
    Boolean playingAgainstAI = false
    Int aiLevel = 2 // 1: Easy, 2: Medium, 3: Hard / Minimax (hope this doesn't lag lol)

    Image xSprite, oSprite

    // Helper: Safely load assets without throwing NullPointer exceptions on the lab computer
    Function loadAssetSafely(fileName)
        String[] possiblePaths = [
            fileName,
            "C:\TicTacToe\" + fileName,
```

```
            "C:\Java Project Files\" + fileName,
            workingDir + fileName
        ]

        For each path in possiblePaths
            If file exists at path Then
                Return Image from path

        // Fallback to resource stream just in case
        Try
            Return loadResourceFromClassPath("/" + fileName)
        Catch
            Print "Warning: Couldn't find " + fileName + ", falling back to plain text X/O."
            Return null
        End Try
    End Function

    // Constructor: Set up everything when the program launches
    Function TicTacToeApp()
        mainWindow = New Frame("Tic-Tac-Toe - Group 25")
        mainWindow.setMaximized(true) // fullscreen looks cleaner for our presentation

        // Load custom icons (try-catch so it doesn't crash if someone deletes the JPEGs)
        Try
            xSprite = removeWhiteBackground(loadAssetSafely("x_icon.jpg"))
            oSprite = removeWhiteBackground(loadAssetSafeless("o_icon.jpg")) // wait typo fix: loadAssetSafely
        Catch
            // Ignore asset errors, text labels will handle it
```

```
    End Try

    screenManager = New CardLayout()
    mainContainer = New Panel(screenManager)

    setupMainMenu()
    setupTokenSelectMenu()
    setupDifficultyMenu()
    setupGameGridScreen()

    // Add cards to the main container panel
    mainContainer.add(menuScreen, "MENU")
    mainContainer.add(tokenScreen, "TOKEN_SELECT")
    mainContainer.add(diffScreen, "DIFFICULTY_SELECT")
    mainContainer.add(boardScreen, "GAME")

    mainWindow.setContentPane(mainContainer)
    screenManager.show(mainContainer, "MENU")
End Function

// Quick helper to make buttons look modern (flat & borderless)
Function styleFlatButton(b)
    b.setOpaque(false)
    b.setContentAreaFilled(false)
    b.setBorderPainted(false)
    b.setFocusPainted(false)
End Function

// 1. Main Menu Screen Setup
```

```
Function setupMainMenu()
    menuScreen = New Panel(GridBagLayout) // using gridbag because manual bounds are trash

    Button twoPlayerBtn = New Button("2 Player Mode")
    Button aiModeBtn = New Button("Play vs Computer")

    styleFlatButton(twoPlayerBtn)
    styleFlatButton(aiModeBtn)

    twoPlayerBtn.onAction ->
        playingAgainstAI = false
        screenManager.show(mainContainer, "TOKEN_SELECT")

    aiModeBtn.onAction ->
        playingAgainstAI = true
        screenManager.show(mainContainer, "DIFFICULTY_SELECT")

    // Add buttons to menu panel with some padding
End Function

// 2. Token Selection (For 2 Player mode)
Function setupTokenSelectMenu()
    tokenScreen = New Panel()
    Button chooseX = New Button("Player 1 is X")
    Button chooseO = New Button("Player 1 is O")

    styleFlatButton(chooseX)
    styleFlatButton(chooseO)
```

```
    chooseX.onAction ->
        clearAndResetBoard()
        playerXTurn = true
        infoLabel.setText("Player X's Turn")
        screenManager.show(mainContainer, "GAME")

    chooseO.onAction ->
        clearAndResetBoard()
        playerXTurn = false
        infoLabel.setText("Player O's Turn")
        screenManager.show(mainContainer, "GAME")
End Function

// 3. AI Difficulty Screen Setup
Function setupDifficultyMenu()
    diffScreen = New Panel()
    Button easyBtn = New Button("Easy (Random)")
    Button medBtn = New Button("Medium (Smart Block)")
    Button hardBtn = New Button("Hard (Minimax Algorithm)")

    styleFlatButton(easyBtn)
    styleFlatButton(medBtn)
    styleFlatButton(hardBtn)

    easyBtn.onAction -> aiLevel = 1; clearAndResetBoard(); screenManager.show(mainContainer, "GAME")
    medBtn.onAction -> aiLevel = 2; clearAndResetBoard(); screenManager.show(mainContainer, "GAME")
```

```
    hardBtn.onAction -> aiLevel = 3; clearAndResetBoard(); screenManager.show(mainContainer, "GAME")
End Function

// 4. Main Game Screen Setup (3x3 Grid)
Function setupGameGridScreen()
    boardScreen = New Panel(BorderLayout)

    // Info panel at top
    Panel topInfo = New Panel(GridLayout(2, 1))
    infoLabel = New Label("Player X's Turn", CENTER)
    scoreTrackerLabel = New Label("X Wins: 0 | O Wins: 0 | Draws: 0", CENTER)
    topInfo.add(infoLabel)
    topInfo.add(scoreTrackerLabel)
    boardScreen.add(topInfo, TOP)

    // 3x3 Grid of buttons
    Panel gridPanel = New Panel(GridLayout(3, 3, 5, 5)) // 5px gap looks nice
    For r = 0 to 2
        For c = 0 to 2
            boardButtons[r][c] = New Button()
            boardButtons[r][c].setFont(Arial, BOLD, 50)

            // capture local variables for lambda click listener
            finalRow = r
            finalCol = c
            boardButtons[r][c].onAction -> handleCellClick(finalRow, finalCol)

            gridPanel.add(boardButtons[r][c])
```

```
            stateGrid[r][c] = ""
        End For
    End For
    boardScreen.add(gridPanel, CENTER)

    // Bottom buttons (New Game & Back to Menu)
    Panel bottomControls = New Panel()
    Button resetBtn = New Button("New Game")
    Button menuBtn = New Button("Main Menu")

    resetBtn.onAction -> clearAndResetBoard()
    menuBtn.onAction -> screenManager.show(mainContainer, "MENU")

    bottomControls.add(resetBtn)
    bottomControls.add(menuBtn)
    boardScreen.add(bottomControls, BOTTOM)
End Function

// Handle what happens when a user clicks a square
Function handleCellClick(r, c)
    If not isGameRunning or stateGrid[r][c] is not empty Then Return

    String activePlayer = playerXTurn ? "X" : "O"
    executePlayerMove(r, c, activePlayer)

    If not isGameRunning Then Return

    // If playing AI and it's now O's turn, trigger the computer move with a tiny delay (feels more natural)
```

```
    If playingAgainstAI and not playerXTurn and isGameRunning Then
        Start Timer (350ms delay) ->
            If not isGameRunning Then Return

            Integer[] chosenMove = null
            If aiLevel == 1 Then
                chosenMove = getRandomMove()
            Else If aiLevel == 2 Then
                chosenMove = getMediumMove() // checks win/block first
            Else
                chosenMove = getMinimaxBestMove() // unbeatable AI
            End If

            If chosenMove is not null Then
                executePlayerMove(chosenMove[0], chosenMove[1], "O")
            End If
        End Timer
    End If
End Function

// Place the move on the board and update UI
Function executePlayerMove(r, c, playerSymbol)
    stateGrid[r][c] = playerSymbol
    Button targetButton = boardButtons[r][c]

    If playerSymbol == "X" Then
        If xSprite is not null Then
            targetButton.setIcon(resizeImage(xSprite, targetButton.width, targetButton.height))
        Else
```

```
            targetButton.setText("X")
        End If
        infoLabel.setText("Player O's Turn")
    Else
        If oSprite is not null Then
            targetButton.setIcon(resizeImage(oSprite, targetButton.width, targetButton.height))
        Else
            targetButton.setText("O")
        End If
        infoLabel.setText("Player X's Turn")
    End If

    evaluateGameStatus()
    playerXTurn = not playerXTurn
End Function

// --- AI STRATEGY LOGIC ---

Function getRandomMove()
    List emptyCells = []
    For r = 0 to 2
        For c = 0 to 2
            If stateGrid[r][c] == "" Then emptyCells.add([r, c])
    If emptyCells.isEmpty() Then Return null
    Return random element from emptyCells
End Function

Function getMediumMove()
    // 1. Can AI win in one move? Do it.
```

```
    // 2. Can player win in next move? Block them.
    // 3. Otherwise pick random.
    Integer[] smartSpot = findImmediateWinOrBlock()
    If smartSpot is not null Then Return smartSpot
    Return getRandomMove()
End Function

Function findImmediateWinOrBlock()
    // Check AI winning move ("O")
    For r = 0 to 2
        For c = 0 to 2
            If stateGrid[r][c] == "" Then
                stateGrid[r][c] = "O"
                If checkWinCondition(stateGrid, "O") Then
                    stateGrid[r][c] = ""
                    Return [r, c]
                End If
                stateGrid[r][c] = ""
            End If
    End For

    // Check player blocking move ("X")
    For r = 0 to 2
        For c = 0 to 2
            If stateGrid[r][c] == "" Then
                stateGrid[r][c] = "X"
                If checkWinCondition(stateGrid, "X") Then
                    stateGrid[r][c] = ""
                    Return [r, c]
```

```
            End If
            stateGrid[r][c] = ""
        End If
    End For

    Return null
End Function

Function getMinimaxBestMove()
    Integer bestScore = -99999
    Integer[] bestMove = null

    For r = 0 to 2
        For c = 0 to 2
            If stateGrid[r][c] == "" Then
                stateGrid[r][c] = "O"
                Integer score = minimaxRecursive(stateGrid, 0, false)
                stateGrid[r][c] = ""

                If score > bestScore Then
                    bestScore = score
                    bestMove = [r, c]
                End If
            End If
    End For
    Return bestMove
End Function

Function minimaxRecursive(boardCopy, depth, isMaximizing)
```

```
If checkWinCondition(boardCopy, "O") Then Return 10 - depth
If checkWinCondition(boardCopy, "X") Then Return depth - 10
If isBoardCompletelyFull(boardCopy) Then Return 0

If isMaximizing Then
    Integer maxEval = -99999
    For r = 0 to 2
        For c = 0 to 2
            If boardCopy[r][c] == "" Then
                boardCopy[r][c] = "O"
                Integer eval = minimaxRecursive(boardCopy, depth + 1, false)
                boardCopy[r][c] = ""
                maxEval = Max(maxEval, eval)
            End If
    End For
    Return maxEval
Else
    Integer minEval = 99999
    For r = 0 to 2
        For c = 0 to 2
            If boardCopy[r][c] == "" Then
                boardCopy[r][c] = "X"
                Integer eval = minimaxRecursive(boardCopy, depth + 1, true)
                boardCopy[r][c] = ""
                minEval = Min(minEval, eval)
            End If
    End For
    Return minEval
End If
```

```
End Function

// --- WIN & STATE VALIDATION ---

Function checkWinCondition(board, symbol)
    // Check rows and columns
    For i = 0 to 2
        If board[i][0] == symbol and board[i][1] == symbol and board[i][2] == symbol Then Return true
        If board[0][i] == symbol and board[1][i] == symbol and board[2][i] == symbol Then Return true
    End For
    // Check diagonals
    If board[0][0] == symbol and board[1][1] == symbol and board[2][2] == symbol Then Return true
    If board[0][2] == symbol and board[1][1] == symbol and board[2][0] == symbol Then Return true

    Return false
End Function

Function isBoardCompletelyFull(board)
    For r = 0 to 2
        For c = 0 to 2
            If board[r][c] == "" Then Return false
    Return true
End Function

Function evaluateGameStatus()
    If checkWinCondition(stateGrid, "X") Then
```

```
            winsX++
            infoLabel.setText("Player X Wins!")
            updateScoreTextDisplay()
            isGameRunning = false
        Else If checkWinCondition(stateGrid, "O") Then
            winsO++
            infoLabel.setText("Player O Wins!")
            updateScoreTextDisplay()
            isGameRunning = false
        Else If isBoardCompletelyFull(stateGrid) Then
            ties++
            infoLabel.setText("It's a Draw!")
            updateScoreTextDisplay()
            isGameRunning = false
        End If
    End Function

    Function updateScoreTextDisplay()
        scoreTrackerLabel.setText("X Wins: " + winsX + " | O Wins: " + winsO + " | Draws: " +
ties)
    End Function

    Function clearAndResetBoard()
        For r = 0 to 2
            For c = 0 to 2
                stateGrid[r][c] = ""
                boardButtons[r][c].setIcon(null)
                boardButtons[r][c].setText("")
            End For
```

```
        End For
        playerXTurn = true
        isGameRunning = true
        infoLabel.setText("Player X's Turn")
    End Function

    // --- UTILS: Image Processing ---
    Function removeWhiteBackground(img)
        // converts white/grey background pixels to transparent ARGB pixels
        // (took forever to get working correctly in our lab sessions)
        Return processedImage
    End Function

    Function resizeImage(img, targetWidth, targetHeight)
        dimension = Min(targetWidth, targetHeight) - 4
        If dimension < 30 Then dimension = 80
        Return scaledImageIcon
    End Function

    // Program Entry Point
    Function main(args)
        Run on SwingUtilities.invokeLater ->
            TicTacToeApp app = New TicTacToeApp()
            app.mainWindow.setVisible(true)
        End Run
    End Function
End Class
```